

Vectorization (Part 2)

How that magic trick (the fastness of vectorization actually works).

w/o vectorization

- for j in range(0, 16):
 $f = f + w[j] * x[j]$

to: $f + w[0] * x[0]$

$t_1: f + w[1] * x[1]$

$\vdots$

$t_{15} = f + w[15] * x[15]$

with vectorization

- $\text{np.dot}(w, x)$

to:

$w[0]$	$w[1]$	$\dots$	$w[15]$
--------	--------	---------	---------

in parallel $*$ $*$ $*$ $*$

$x[0]$	$x[1]$	$\dots$	$x[15]$
--------	--------	---------	---------

$\swarrow \quad \downarrow \quad \quad \downarrow$
 $w[0] * x[0] + w[1] * x[1] + \dots + w[15] * x[15]$

The computer takes these 16 numbers & adds using specialized hardware to add them all together efficiently rather than distinct additions.

efficient $\rightarrow$ scale to large datasets

Now,

concrete example of how this helps in implementing multiple linear regression:-

Say you have a problem with 16 features & 16 params in addition to parameter b .

Gradient Descent

$$\vec{w} = (w_1, w_2, \dots, w_{16}) \quad b, \quad \vec{d} = (d_1, d_2, \dots, d_{16})$$

You calculate 16 derivatives for these 16 weights & codes in code you stored it as:

1) $w = \text{np.array}([0.5, 1.3, \dots, 3.4])$

2) $d = \text{np.array}([0.3, 0.2, \dots, 0.4])$

For this example ignore b :

Now you wanna compute,

• $w_j = w_j - 0.1 d_j$ for $j = 1 \dots 16$

you would be doing

w/o vectorization

$$w_1 = w_1 - 0.1 d_1$$

$$w_2 = w_2 - 0.1 d_2$$

⋮

$$w_{16} = w_{16} - 0.1 d_{16}$$

Code:

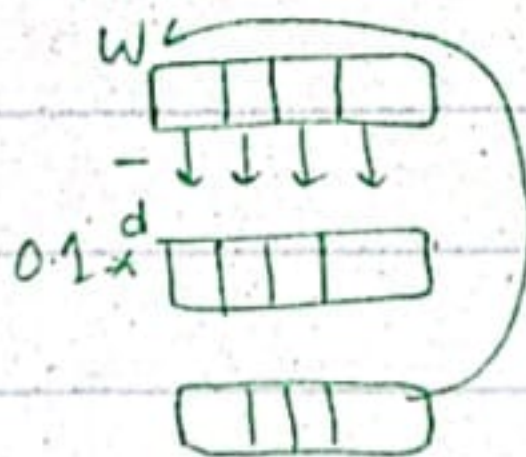
```
for j in range(0, 16):
```

```
    w[j] = w[j] - 0.1 * d[j]
```

with vectorization

$$\vec{w} = \vec{w} - 0.1 \vec{d}$$

takes all 16 values



all at same time

Code:

```
w = w - 0.1 * d
```